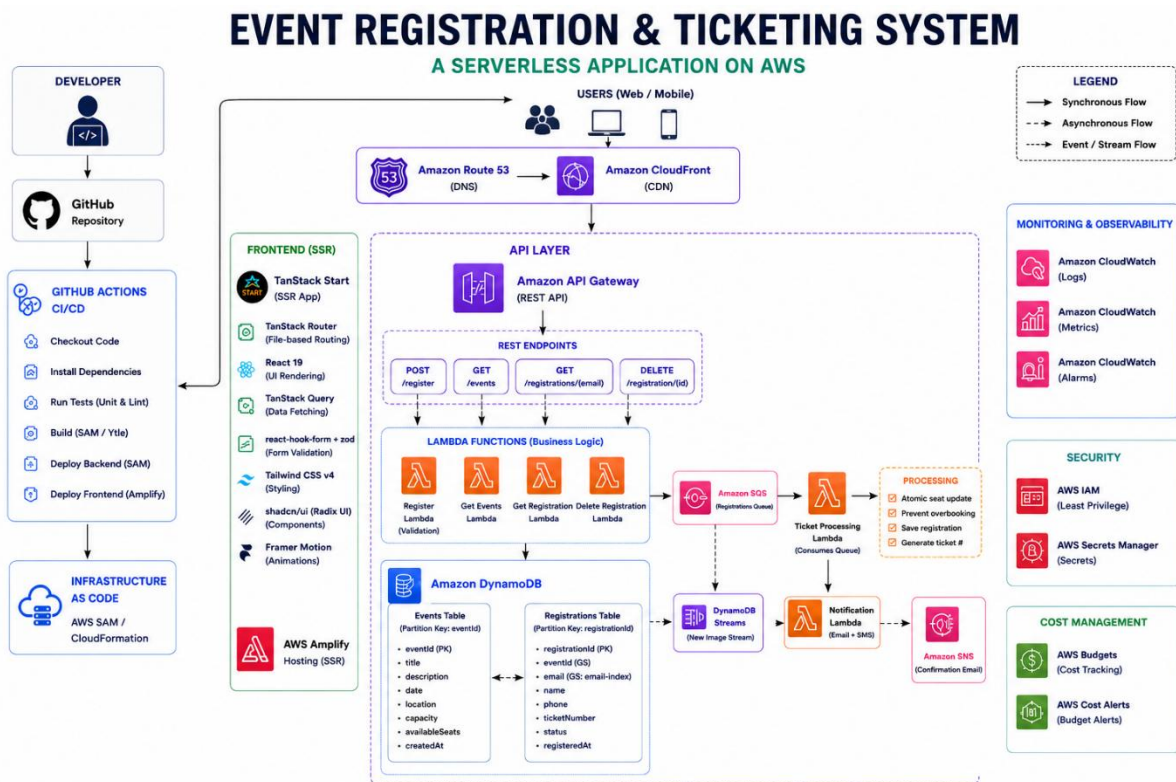


Event Registration & Ticketing System Architecture & End-to-End Data Flow



High-Level System Architecture

The Event Registration & Ticketing System is a cloud-native serverless web application built entirely on Amazon Web Services (AWS). The architecture follows a loosely coupled, event-driven design that separates the presentation, application, processing, and data layers into independent services. This design allows each component to scale automatically based on demand, improves fault tolerance, reduces operational overhead, and enables continuous deployment without managing traditional servers. The application consists of a Server-Side Rendered (SSR) frontend built with TanStack Start, a RESTful backend powered by Amazon API Gateway and AWS Lambda, a NoSQL database using Amazon DynamoDB, asynchronous processing with Amazon SQS, automated notifications using Amazon SNS, and a fully automated deployment pipeline using GitHub Actions and AWS SAM.

The frontend is developed using **TanStack Start**, which provides a modern full-stack React framework capable of Server-Side Rendering (SSR). It uses React 19 for building interactive user interfaces, TanStack Router for file-based routing, TanStack Query for efficient client-server data synchronization, Tailwind CSS for responsive styling, React Hook Form with Zod for form validation, Shadcn/UI for reusable interface components, and Framer Motion for

smooth animations. The frontend is hosted on **AWS Amplify**, which automatically builds and deploys the application whenever new code is pushed to GitHub. Amplify also handles SSR hosting, ensuring that pages are rendered on the server before being delivered to users, resulting in faster page loads and improved search engine optimization.

User requests first pass through **Amazon Route 53**, which resolves the application's domain name into the correct CloudFront distribution endpoint. **Amazon CloudFront** then serves as the global Content Delivery Network (CDN), caching static assets such as JavaScript bundles, CSS files, images, and fonts at edge locations around the world.

The backend is exposed through **Amazon API Gateway**, which serves as the single entry point for all REST API requests. API Gateway manages request routing, throttling, request validation, authentication integration, and secure HTTPS communication. Each REST endpoint invokes a dedicated AWS Lambda function responsible for executing a specific piece of business logic. This function-per-feature architecture keeps services modular, independently deployable, and easy to maintain.

Business logic is implemented using several **AWS Lambda** functions. The **Register** Lambda validates incoming registration requests, verifies event availability, and creates new registrations. The **Get Events** Lambda retrieves all available events from the database, while the **Get Registration** Lambda searches for registration information using an email address. The **Delete Registration** Lambda removes existing registrations when cancellation requests are received. Because each Lambda function performs a single responsibility, the application remains highly maintainable and can scale individual functions independently based on workload.

Application data is stored in **Amazon DynamoDB**, a fully managed NoSQL. The architecture separates data into two logical tables. The **Events table** stores event information such as titles, dates, locations, capacities, and remaining seats using the event ID as its partition key. The **Registrations table** stores attendee information using the registration ID as its partition key and a Global Secondary Index (GSI) on email to support efficient searches by registered email address. DynamoDB automatically scales storage and throughput without requiring manual database administration.

To ensure registration processing remains reliable during periods of high demand, the architecture introduces **Amazon Simple Queue Service (Amazon SQS)** between the registration service and ticket processing. Instead of performing all operations synchronously, successful registrations are placed into the queue, allowing ticket generation to occur asynchronously. This decouples the user-facing registration process from downstream processing, preventing delays during traffic spikes while ensuring that no registration requests are lost if downstream services become temporarily unavailable.

The **Ticket Processing Lambda** continuously polls Amazon SQS for new messages. For each registration request received, it performs additional processing such as validating seat availability, preventing duplicate reservations, finalizing the registration, and generating a unique ticket number. By separating ticket generation from the initial registration request, the application maintains a responsive user experience even when processing thousands of simultaneous registrations.

Whenever the Registrations table is updated, **DynamoDB Streams** automatically captures the change event and triggers the **Notification Lambda**. Rather than polling the database for updates, the notification service reacts immediately whenever new records are inserted or modified. The Notification Lambda prepares the confirmation message and publishes it to **Amazon SNS**, which distributes registration confirmation notifications to subscribers. This event-driven architecture ensures that notifications remain independent of the registration workflow while providing reliable and scalable message delivery.

Operational visibility is provided through **Amazon CloudWatch**, which collects logs, metrics, and alarms from every Lambda function and API request. CloudWatch enables developers to monitor invocation counts, execution duration, failures, throttling events, and overall application health in real time. Automated alarms can notify administrators whenever predefined operational thresholds are exceeded, allowing rapid detection and resolution of issues before they affect users.

Security throughout the application is enforced using **AWS Identity and Access Management (IAM)** and **AWS Secrets Manager**. IAM roles grant each AWS service only the permissions required to perform its specific tasks, following the principle of least privilege. Lambda functions receive execution roles that restrict access to only the DynamoDB tables, SQS queues, SNS topics, and CloudWatch resources they require. AWS Secrets Manager securely stores sensitive configuration values and credentials, eliminating the need to embed secrets directly within the application's source code.

Infrastructure provisioning and application deployment are fully automated using **AWS Serverless Application Model (AWS SAM)** and **AWS CloudFormation**. Infrastructure is defined as code, allowing API Gateway resources, Lambda functions, IAM roles, DynamoDB tables, SQS queues, SNS topics, and supporting resources to be created consistently across development, testing, and production environments.

Step-by-Step Request Flow (User Journey)

When a user opens the Event Registration & Ticketing System in a web browser, the browser first performs a DNS lookup for the application's domain name. **Amazon Route 53** resolves the domain and directs the request to the appropriate **Amazon CloudFront** distribution. Because CloudFront maintains edge locations across the globe, static assets such as JavaScript files, CSS stylesheets, fonts, and images are delivered from the nearest edge location whenever possible. If the requested content is not already cached, CloudFront retrieves it from the application's AWS Amplify hosting environment before caching it for future requests.

AWS Amplify serves the TanStack Start application and performs Server-Side Rendering (SSR) for dynamic pages. Instead of sending an empty HTML page to the browser, the server renders the initial React components before transmitting the response. This reduces the time required for users to view meaningful content and improves both perceived performance and search engine indexing.

Once the application loads in the browser, the frontend begins communicating with the backend whenever users browse available events, register for an event, search for existing registrations, or cancel a reservation. These requests are sent securely over HTTPS to **Amazon API Gateway**, which validates the incoming request and forwards it to the appropriate AWS Lambda function based on the configured REST endpoint.

If a user requests the list of available events, API Gateway invokes the **Get Events Lambda**. The Lambda function queries the Events table in Amazon DynamoDB, retrieves all active event records, formats the response as JSON, and returns the data through API Gateway to the frontend, where the event listings are rendered dynamically.

During event registration, the frontend submits the completed registration form to the **Register Lambda** through API Gateway. The Lambda function first validates the submitted information, verifies that the selected event exists, and confirms that registration is still available. After successful validation, the registration details are stored in the Registrations table within Amazon DynamoDB. Rather than generating tickets immediately, the Lambda places a message containing the registration details into Amazon SQS before returning a successful response to the user. Because ticket processing occurs asynchronously, users receive immediate confirmation that their registration has been accepted without waiting for additional processing to complete.

The **Ticket Processing Lambda** continuously consumes messages from the Amazon SQS queue. Each queued registration is processed independently, ensuring that temporary spikes in user activity do not overwhelm downstream services. The Lambda verifies seat allocation, prevents overbooking, finalizes the registration, and generates a unique ticket identifier before completing processing.

As soon as the registration record is updated, **DynamoDB Streams** automatically emits a stream event describing the database change. This event immediately invokes the **Notification Lambda**, which extracts the registration information, constructs a confirmation message, and publishes it to **Amazon SNS**. SNS then distributes the confirmation notification to all subscribed endpoints, ensuring reliable delivery without delaying the registration workflow.

Throughout this entire process, **Amazon CloudWatch** continuously records logs, execution metrics, API response times, Lambda durations, and operational events. These metrics allow administrators to monitor application performance, identify bottlenecks, troubleshoot failures, and maintain overall system reliability.

CI/CD Pipeline Flow (Developer Journey)

Application development begins when a developer implements new features or fixes bugs within the local development environment. After testing the changes locally, the developer commits the updated source code and pushes it to the project's GitHub repository. Every push to the **main** branch automatically triggers the configured **GitHub Actions** workflow, initiating the Continuous Integration and Continuous Deployment (CI/CD) pipeline.

The workflow begins by checking out the latest version of the repository before installing all required frontend and backend dependencies. The pipeline then executes automated quality assurance tasks such as dependency installation, project compilation, linting, and unit testing. Performing these validation steps before deployment ensures that syntax errors, failed tests, and build issues are detected early, preventing unstable code from reaching production.

After the application successfully passes all validation stages, GitHub Actions securely authenticates with AWS using configured credentials stored as GitHub Secrets or through an IAM role using OpenID Connect (OIDC). This authentication mechanism provides temporary credentials during deployment while avoiding the need to store long-lived AWS access keys within the repository.

Once authenticated, the backend deployment begins using **AWS SAM**. SAM packages the Lambda functions, generates the CloudFormation deployment package, uploads application artifacts, and deploys infrastructure changes using AWS CloudFormation. CloudFormation compares the desired infrastructure state with the currently deployed resources and applies only the required changes, allowing infrastructure updates to occur predictably and consistently.

Following the backend deployment, GitHub Actions builds the TanStack Start frontend and deploys the generated application to **AWS Amplify**. Amplify automatically updates the hosting environment and prepares the latest server-rendered application version for production. Once deployment is complete, **Amazon CloudFront** begins serving the updated application to users. Cached content is refreshed as necessary, ensuring that users receive the latest application version while minimizing service interruption.

Because AWS Lambda functions are inherently stateless and deployed independently, updates occur without requiring server maintenance or application downtime. Existing function instances continue serving active requests while new versions are introduced, allowing deployments to complete seamlessly with minimal impact on end users.

Security & Data Isolation

Security within the Event Registration & Ticketing System is based on AWS's shared responsibility model and follows the principle of least privilege. Every AWS service operates with a dedicated IAM role containing only the permissions necessary to perform its assigned responsibilities. API Gateway is permitted to invoke Lambda functions, Lambda functions receive permission to access only their required DynamoDB tables, SQS queues, SNS topics, and CloudWatch resources, while deployment permissions are restricted exclusively to the GitHub Actions deployment workflow. This granular access control significantly reduces the potential impact of compromised credentials or unauthorized actions.

Sensitive configuration values are securely stored in **AWS Secrets Manager**, ensuring that secrets such as API credentials and configuration values are never embedded directly within the application's source code or deployment artifacts. Lambda functions retrieve secrets securely at runtime using their assigned IAM permissions, allowing credentials to be rotated without requiring application code changes.

All communication between users and the application occurs over encrypted HTTPS connections. Amazon CloudFront serves as the public entry point and works together with API Gateway to ensure that traffic remains encrypted while in transit. Encryption protects sensitive registration information from interception and guarantees the confidentiality and integrity of data exchanged between users and AWS services.

Data isolation is achieved through the use of managed AWS services rather than directly exposing backend infrastructure. Users never communicate directly with DynamoDB, Lambda, or Amazon SQS. Instead, every request passes through API Gateway, which validates and securely routes requests to the appropriate backend function. DynamoDB accepts requests only from authorized Lambda execution roles, while Amazon SQS and Amazon SNS similarly restrict access through IAM policies. This layered architecture ensures that each service communicates only with explicitly authorized components, creating a secure, resilient, and highly scalable serverless application that adheres to AWS architectural best practices.